

Day 4: File Handling & Modules

AI and Machine Learning Course

Sandesh Dhital

Nesfield International College

July 23, 2026

Today's Agenda

- 1 File Handling
- 2 CSV Files
- 3 Exception Handling
- 4 Modules
- 5 Practice

Why File Handling?

- Programs need to **persist data** beyond runtime
- Read configuration, datasets, logs
- Write results, reports, processed data
- Essential for **data science and ML pipelines**

Common File Types

Text files (.txt), CSV files (.csv), JSON (.json), Binary files

Opening and Closing Files

```
# Basic file opening
file = open("data.txt", "r") # Open for reading
content = file.read()
file.close() # Always close!

# Better approach: using 'with' statement
with open("data.txt", "r") as file:
    content = file.read()
# File automatically closed here
```

Important

Always use with statement – it ensures the file is properly closed even if an error occurs.

File Open Modes

Mode	Description
"r"	Read (default) – file must exist
"w"	Write – creates new or overwrites existing
"a"	Append – adds to end of file
"x"	Exclusive create – fails if file exists
"r+"	Read and write
"b"	Binary mode (e.g., "rb", "wb")

Reading Text Files

```
# Read entire file as one string
with open("poem.txt", "r") as f:
    content = f.read()
    print(content)

# Read line by line (memory efficient)
with open("poem.txt", "r") as f:
    for line in f:
        print(line.strip())

# Read all lines into a list
with open("poem.txt", "r") as f:
    lines = f.readlines()
    print(lines[0]) # First line
```

Writing Text Files

```
# Writing to a file
with open("output.txt", "w") as f:
    f.write("Hello, World!\n")
    f.write("Second line\n")

# Writing multiple lines
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("output.txt", "w") as f:
    f.writelines(lines)

# Appending to a file
with open("log.txt", "a") as f:
    f.write("New log entry\n")
```

Understanding CSV Files

- **CSV** = Comma-Separated Values
- A simple, widely-used format for storing **tabular data**
- Each line represents a **row**; columns are separated by commas
- First row is usually the **header** (column names)
- Can be opened in Excel, Google Sheets, or any text editor

Example: students.csv

```
name, age, grade  
Alice, 20, A  
Bob, 22, B
```

Why CSV? Lightweight, human-readable, and universally supported — ideal for ML datasets.

Reading CSV Files

```
import csv

# Reading CSV
with open("students.csv", "r") as f:
    reader = csv.reader(f)
    header = next(reader) # Skip header
    for row in reader:
        print(row) # Each row is a list

# Reading CSV as dictionaries
with open("students.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["grade"])
```

Writing CSV Files

```
import csv

# Writing CSV
header = ["name", "age", "grade"]
students = [
    ["Alice", 20, "A"],
    ["Bob", 22, "B"],
    ["Charlie", 21, "A"]
]

with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(header)
    writer.writerows(students)
```

Note

Use `newline=""` on Windows to prevent extra blank lines in CSV.

What Are Exceptions?

- Errors that occur **during program execution**
- Without handling, they **crash** the program
- Python provides try/except to handle them gracefully

```
# This crashes!  
num = int("hello")    # ValueError  
  
# This crashes too!  
file = open("nonexistent.txt") # FileNotFoundError
```

Try / Except Blocks

```
try:
    file = open("data.txt", "r")
    content = file.read()
    number = int(content)
except FileNotFoundError:
    print("File does not exist!")
except ValueError:
    print("File doesn't contain a valid number!")
except Exception as e:
    print(f"Unexpected error: {e}")
finally:
    print("This always runs!")
```

Structure: try → except → else (no error) → finally (always)

Safe File Reading Pattern

```
def read_file_safe(filename):  
    try:  
        with open(filename, "r") as f:  
            return f.read()  
    except FileNotFoundError:  
        print(f"Error: '{filename}' not found.")  
        return None  
    except PermissionError:  
        print(f"Error: No permission to read.")  
        return None  
  
content = read_file_safe("mydata.txt")  
if content:  
    print(content)
```

Understanding Modules

- A **module** is simply a .py file containing Python code
- Modules help **organize** code into reusable, logical units
- Python has thousands of **built-in** and **third-party** modules
- You can also create your **own modules**

Benefits of Using Modules

- ① **Code Reusability** – Write once, use everywhere
- ② **Organization** – Break large programs into manageable files
- ③ **Namespace Management** – Avoid naming conflicts

Popular ML Modules: numpy, pandas, scikit-learn, matplotlib

Importing Modules

```
# Import entire module
import math
print(math.sqrt(16))    # 4.0

# Import specific functions
from math import sqrt, pi
print(sqrt(16))        # 4.0

# Import with alias
import numpy as np
arr = np.array([1, 2, 3])

# Import everything (not recommended)
from math import *
```

Creating Your Own Module

myutils.py

```
# myutils.py
def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b

PI = 3.14159
```

main.py

```
# main.py
import myutils

print(myutils.greet("Alice"))
print(myutils.add(3, 5))
print(myutils.PI)

# Or:
from myutils import greet
print(greet("Bob"))
```


Practice: CSV Data Reader

```
import csv

def read_csv_data(filename):
    try:
        with open(filename, "r") as f:
            reader = csv.DictReader(f)
            data = list(reader)
            print(f"Total records: {len(data)}")
            print(f"Columns: {list(data[0].keys())}")
            return data
    except FileNotFoundError:
        print(f"Error: {filename} not found!")
        return []
    except Exception as e:
        print(f"Error reading CSV: {e}")
        return []

students = read_csv_data("students.csv")
```

Summary

- 1 **File Handling:** Use `with open()` for safe file operations
- 2 **Text Files:** `read()`, `readline()`, `write()`
- 3 **CSV Files:** Use `csv` module for structured data
- 4 **Exception Handling:** `try/except` prevents crashes
- 5 **Modules:** Organize code, reuse with `import`

Homework

Create a program that reads a CSV file of student grades, calculates the average, and writes the results to a new file. Handle all possible errors!